

Terraform Complete Step-by-Step Lab Document

Goal

This lab document will take you from Terraform basics to advanced production-style usage using Azure as the main cloud platform.

You will learn:

- Terraform installation
- Azure CLI login
- Provider configuration
- Resource Group creation
- Variables
- terraform.tfvars
- Outputs
- State file
- Remote backend using Azure Storage
- Lifecycle rules
- count and for_each
- Modules
- Import existing resources
- Drift detection
- Destroy protection
- CI/CD ready project structure

LAB 0: Prerequisites

Required Tools

Install these tools on Windows 11:

1. Terraform
2. Azure CLI
3. Visual Studio Code
4. Git
5. PowerShell

Check Installation

Open PowerShell and run:

```
terraform version  
az version
```

```
git --version  
code --version
```

Expected result:

- Terraform version should show
- Azure CLI version should show
- Git version should show
- VS Code should open or show version

LAB 1: Create Terraform Working Folder

Step 1: Create Folder

```
mkdir D:\terraform-labs  
cd D:\terraform-labs  
mkdir lab01-basic-rg  
cd lab01-basic-rg
```

Step 2: Open in VS Code

```
code .
```

LAB 2: Azure Login

Step 1: Login to Azure

```
az login
```

Browser will open. Login with your Azure account.

Step 2: Check Subscription

```
az account show
```

Step 3: If Multiple Subscriptions Exist

List subscriptions:

```
az account list --output table
```

Set correct subscription:

```
az account set --subscription "YOUR_SUBSCRIPTION_ID"
```

Verify again:

```
az account show --output table
```

LAB 3: First Terraform Code - Azure Resource Group

Step 1: Create main.tf

Create a file named:

```
main.tf
```

Add this code:

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name = "terraform-basic-rg"
```

```
    location = "West Europe"  
  }  
}
```

Step 2: Initialize Terraform

```
terraform init
```

What happens:

- Terraform downloads AzureRM provider
- Creates `.terraform` folder
- Creates `.terraform.lock.hcl`

Step 3: Format Code

```
terraform fmt
```

Step 4: Validate Code

```
terraform validate
```

Expected output:

```
Success! The configuration is valid.
```

Step 5: Plan

```
terraform plan
```

You should see:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Step 6: Apply

```
terraform apply
```

Type:

```
yes
```

Step 7: Verify in Azure Portal

Go to Azure Portal and check Resource Groups.

You should see:

```
terraform-basic-rg
```

LAB 4: Understand Terraform Files

After apply, your folder contains:

```
main.tf
terraform.tfstate
terraform.tfstate.backup
.terraform/
.terraform.lock.hcl
```

Important Files

File	Purpose
main.tf	Terraform code
terraform.tfstate	Current infrastructure state
terraform.tfstate.backup	Previous state backup
.terraform	Provider/plugin folder
.terraform.lock.hcl	Provider version lock file

LAB 5: Add Variables

Now we will remove hardcoded values.

Step 1: Create variables.tf

```
variable "resource_group_name" {  
  description = "Name of the Azure Resource Group"  
  type        = string  
}  
  
variable "location" {  
  description = "Azure region"  
  type        = string  
  default     = "West Europe"  
}
```

Step 2: Update main.tf

Replace Resource Group block with:

```
resource "azurerm_resource_group" "rg" {  
  name      = var.resource_group_name  
  location = var.location  
}
```

Step 3: Create terraform.tfvars

```
resource_group_name = "terraform-variable-rg"  
location            = "West Europe"
```

Step 4: Run Plan

```
terraform plan
```

Terraform will show that it wants to replace the resource group because the name changed.

Step 5: Apply

```
terraform apply
```

Type:

```
yes
```

LAB 6: Add Outputs

Step 1: Create outputs.tf

```
output "resource_group_name" {  
  value = azurerm_resource_group.rg.name  
}  
  
output "resource_group_location" {  
  value = azurerm_resource_group.rg.location  
}  
  
output "resource_group_id" {  
  value = azurerm_resource_group.rg.id  
}
```

Step 2: Apply Again

```
terraform apply
```

Step 3: Check Output

```
terraform output
```

LAB 7: Create Azure Storage Account

Step 1: Add Storage Account Resource

Add this in `main.tf`:

```
resource "azurerm_storage_account" "storage" {  
  name                = "satishstoragelab01"  
  resource_group_name = azurerm_resource_group.rg.name  
  location             = azurerm_resource_group.rg.location  
  account_tier         = "Standard"  
  account_replication_type = "LRS"  
}
```

Important:

Storage account name must be globally unique, lowercase, only letters and numbers, and 3 to 24 characters.

If this name fails, use another unique name like:

satishstoragelab2026

Step 2: Run Plan

terraform plan

Expected:

Plan: 1 to add, 0 to change, 0 to destroy.

Step 3: Apply

terraform apply

LAB 8: Implicit Dependency

Terraform understands this dependency automatically:

```
resource_group_name = azurerm_resource_group.rg.name
location            = azurerm_resource_group.rg.location
```

This means:

Storage Account depends on Resource Group.

No need to write `depends_on` in most cases.

LAB 9: Explicit Dependency

Sometimes you want to force dependency manually.

Example:

```
resource "azurerm_storage_account" "storage" {
  name                = "satishstoragelab01"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  account_tier        = "Standard"
  account_replication_type = "LRS"

  depends_on = [azurerm_resource_group.rg]
}
```

Use explicit dependency only when Terraform cannot automatically understand the relationship.

LAB 10: Terraform State Commands

List State Resources

```
terraform state list
```

Expected:

```
azurerm_resource_group.rg
azurerm_storage_account.storage
```

Show Specific Resource

```
terraform state show azurerm_resource_group.rg
```

Show Full Infrastructure

```
terraform show
```

LAB 11: Drift Detection

Drift means infrastructure changed outside Terraform.

Example:

Someone changes Resource Group tag or storage setting manually from Azure Portal.

Step 1: Add Tag in Azure Portal Manually

Go to Azure Portal → Resource Group → Tags.

Add:

```
owner = manual-change
```

Step 2: Run Terraform Plan

```
terraform plan
```

Terraform will detect difference between real infrastructure and Terraform code.

This is called drift detection.

LAB 12: Add Tags Properly in Terraform

Step 1: Add Variable

In `variables.tf`:

```
variable "tags" {  
  description = "Common tags"  
  type        = map(string)  
  default = {  
    environment = "lab"  
    owner       = "satish"  
  }  
}
```

Step 2: Add Tags to Resource Group

```
resource "azurerm_resource_group" "rg" {  
  name      = var.resource_group_name  
  location  = var.location  
  tags      = var.tags  
}
```

Step 3: Add Tags to Storage Account

```
resource "azurerm_storage_account" "storage" {  
  name                        = "satishstoragelab01"  
  resource_group_name        = azurerm_resource_group.rg.name  
  location                   = azurerm_resource_group.rg.location  
  account_tier               = "Standard"  
  account_replication_type   = "LRS"  
  tags                       = var.tags  
}
```

Step 4: Apply

```
terraform apply
```

LAB 13: Lifecycle - prevent_destroy

This protects critical resources from accidental deletion.

Step 1: Add lifecycle to Resource Group

```
resource "azurerm_resource_group" "rg" {
  name      = var.resource_group_name
  location  = var.location
  tags      = var.tags

  lifecycle {
    prevent_destroy = true
  }
}
```

Step 2: Test Destroy

```
terraform destroy
```

Expected:

Terraform should block deletion.

You will see error because `prevent_destroy = true` is enabled.

Important Interview Answer

`prevent_destroy` protects only Terraform-based deletion. It cannot stop someone from deleting the resource directly from Azure Portal if they have permission.

For real protection, use:

- Terraform lifecycle `prevent_destroy`
- Azure RBAC
- Resource Locks
- Approval process in CI/CD

LAB 14: Lifecycle - ignore_changes

Sometimes another team or Azure policy updates tags automatically.

To ignore tag changes:

```
lifecycle {
  ignore_changes = [tags]
}
```

Full example:

```
resource "azurerm_resource_group" "rg" {
  name      = var.resource_group_name
  location  = var.location
  tags      = var.tags

  lifecycle {
    ignore_changes = [tags]
  }
}
```

LAB 15: count

Use count when you need multiple similar resources.

Step 1: Create New Folder

```
cd D:\terraform-labs
mkdir lab02-count
cd lab02-count
```

Step 2: Create main.tf

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

```
resource "azurerm_resource_group" "rg" {  
  count    = 3  
  name     = "count-rg-${count.index}"  
  location = "West Europe"  
}
```

Step 3: Run

```
terraform init  
terraform plan  
terraform apply
```

This creates:

```
count-rg-0  
count-rg-1  
count-rg-2
```

LAB 16: for_each

for_each is better when each resource has a unique name.

Step 1: Create New Folder

```
cd D:\terraform-labs  
mkdir lab03-foreach  
cd lab03-foreach
```

Step 2: Create main.tf

```
terraform {  
  required_providers {  
    azurerm = {  
      source = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}
```

```
provider "azurerm" {  
  features {}  
}  
  
resource "azurerm_resource_group" "rg" {  
  for_each = toset(["dev-rg", "uat-rg", "prod-rg"])  
  
  name      = each.key  
  location = "West Europe"  
}
```

Step 3: Run

```
terraform init  
terraform plan  
terraform apply
```

Important Note

This works:

```
for_each = toset(["dev", "uat", "prod"])
```

This does not work:

```
for_each = toset([1, 2, 3])
```

Because `for_each` supports maps and sets of strings.

LAB 17: Map with `for_each`

Example

```
variable "resource_groups" {  
  type = map(string)  
  default = {  
    dev = "West Europe"  
    uat = "North Europe"  
    prod = "West Europe"  
  }  
}
```

```

}

resource "azurerm_resource_group" "rg" {
  for_each = var.resource_groups

  name      = "${each.key}-rg"
  location = each.value
}

```

Result:

Key	Name	Location
dev	dev-rg	West Europe
uat	uat-rg	North Europe
prod	prod-rg	West Europe

LAB 18: Remote Backend on Azure Storage

This is very important for real production and team work.

Why Remote Backend?

Local state problem:

- Only available on your laptop
- Risk of deletion
- Team conflict
- No central control

Remote backend benefit:

- Central state file
- Team access
- State locking
- CI/CD support

Step 1: Create Backend Resource Group

```
az group create --name tfstate-rg --location westeurope
```


Step 2: Create Storage Account

Storage account name must be unique.

```
az storage account create `
  --name satishterraformstate01 `
  --resource-group tfstate-rg `
  --location westeurope `
  --sku Standard_LRS
```

If name already exists, use another name:

```
satishterraformstate2026
```

Step 3: Create Container

```
az storage container create `
  --name tfstate `
  --account-name satishterraformstate01 `
  --auth-mode login
```

If permission issue comes, use account key method:

```
$key = az storage account keys list `
  --resource-group tfstate-rg `
  --account-name satishterraformstate01 `
  --query '[0].value' `
  --output tsv

az storage container create `
  --name tfstate `
  --account-name satishterraformstate01 `
  --account-key $key
```

Step 4: Add backend.tf

Create file:

```
backend.tf
```

Add:

```
terraform {  
  backend "azurerm" {  
    resource_group_name = "tfstate-rg"  
    storage_account_name = "satishterraformstate01"  
    container_name      = "tfstate"  
    key                  = "lab01.terraform.tfstate"  
  }  
}
```

Step 5: Reinitialize Terraform

```
terraform init
```

Terraform will ask to migrate local state to remote backend.

Type:

```
yes
```

Step 6: Verify State in Azure Portal

Go to:

```
Storage Account → Containers → tfstate
```

You should see:

```
lab01.terraform.tfstate
```

LAB 19: Module Basics

Why Modules?

Modules make Terraform reusable.

Example:

Instead of writing Resource Group code again and again, create one module and reuse it.

Step 1: Create Folder Structure

```
cd D:\terraform-labs
mkdir lab04-modules
cd lab04-modules
mkdir modules
mkdir modules\resource_group
```

Folder structure:

```
lab04-modules/
├─ main.tf
├─ variables.tf
├─ outputs.tf
└─ modules/
    └─ resource_group/
        ├── main.tf
        ├── variables.tf
        └─ outputs.tf
```

Step 2: Module variables.tf

File:

```
modules/resource_group/variables.tf
```

Code:

```
variable "name" {
  type = string
}

variable "location" {
  type = string
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

Step 3: Module main.tf

File:

```
modules/resource_group/main.tf
```

Code:

```
resource "azurerm_resource_group" "this" {  
  name      = var.name  
  location = var.location  
  tags      = var.tags  
}
```

Step 4: Module outputs.tf

File:

```
modules/resource_group/outputs.tf
```

Code:

```
output "name" {  
  value = azurerm_resource_group.this.name  
}  
  
output "id" {  
  value = azurerm_resource_group.this.id  
}
```

Step 5: Root main.tf

File:

```
main.tf
```

Code:

```
terraform {  
  required_providers {
```

```
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}

module "dev_rg" {
  source = "../modules/resource_group"

  name      = "module-dev-rg"
  location = "West Europe"

  tags = {
    environment = "dev"
    owner       = "satish"
  }
}
```

Step 6: Root outputs.tf

```
output "dev_rg_name" {
  value = module.dev_rg.name
}

output "dev_rg_id" {
  value = module.dev_rg.id
}
```

Step 7: Run

```
terraform init
terraform plan
terraform apply
```

LAB 20: Create Multiple Resource Groups Using Module and for_each

Update root `main.tf`:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}

variable "resource_groups" {
  type = map(object({
    location = string
    tags     = map(string)
  }))

  default = {
    dev = {
      location = "West Europe"
      tags = {
        environment = "dev"
        owner       = "satish"
      }
    }

    uat = {
      location = "West Europe"
      tags = {
        environment = "uat"
        owner       = "satish"
      }
    }

    prod = {
      location = "West Europe"
      tags = {
        environment = "prod"
        owner       = "satish"
      }
    }
  }
}
```

```
    }  
  }  
  
  module "resource_groups" {  
    source = "./modules/resource_group"  
  
    for_each = var.resource_groups  
  
    name      = "${each.key}-module-rg"  
    location = each.value.location  
    tags      = each.value.tags  
  }  
}
```

Run:

```
terraform plan  
terraform apply
```

LAB 21: Terraform Import Existing Azure Resource

Import is used when a resource already exists in Azure but is not managed by Terraform.

Step 1: Create Resource Group Manually

Azure Portal or CLI:

```
az group create --name manual-import-rg --location westeurope
```

Step 2: Write Terraform Resource Block

```
resource "azurerm_resource_group" "imported_rg" {  
  name      = "manual-import-rg"  
  location = "West Europe"  
}
```

Step 3: Get Resource ID

```
az group show --name manual-import-rg --query id --output tsv
```

Example output:

```
/subscriptions/xxxx/resourceGroups/manual-import-rg
```

Step 4: Import

```
terraform import azurerm_resource_group.imported_rg "/subscriptions/xxxx/  
resourceGroups/manual-import-rg"
```

Step 5: Plan

```
terraform plan
```

If code matches real resource, Terraform should show:

```
No changes.
```

LAB 22: Remove Resource from State Without Deleting Cloud Resource

Sometimes you want Terraform to stop managing a resource but not delete it.

Command:

```
terraform state rm azurerm_resource_group.imported_rg
```

This removes from Terraform state only.

It does not delete Azure resource.

LAB 23: Targeted Plan and Apply

Use target when you want to apply only one resource.

```
terraform plan -target=azurerm_resource_group.rg
terraform apply -target=azurerm_resource_group.rg
```

Warning:

Use `-target` carefully. It is useful for troubleshooting but not recommended as daily production practice.

LAB 24: Terraform Destroy

Destroy all resources managed by current state:

```
terraform destroy
```

Destroy with auto approve:

```
terraform destroy -auto-approve
```

Destroy specific resource:

```
terraform destroy -target=azurerm_storage_account.storage
```

LAB 25: Production Folder Structure

Recommended real-market structure:

```
terraform-enterprise-project/
├─ modules/
│   ├─ resource_group/
│   ├─ storage_account/
│   ├─ network/
│   ├─ vm/
│   └─ aks/
├─ environments/
│   └─ dev/
```

```
| | | | | main.tf
| | | | | variables.tf
| | | | | terraform.tfvars
| | | | | backend.tf
| | | | | outputs.tf
| | | | |
| | | | | uat/
| | | | | prod/
| | | | |
| | | | | README.md
```

LAB 26: Dev / UAT / Prod Concept

Dev

Used by developers.

UAT

Used by testers/business users.

Prod

Live production environment.

Best practice:

- Separate state file for each environment
- Separate tfvars for each environment
- Approval before prod apply

LAB 27: Environment Specific Backend

Dev backend:

```
key = "dev.terraform.tfstate"
```

UAT backend:

```
key = "uat.terraform.tfstate"
```

Prod backend:

```
key = "prod.terraform.tfstate"
```

LAB 28: Terraform with Jenkins Pipeline Concept

Pipeline flow:

```
Developer → Git Push → Jenkins → terraform fmt → validate → plan → approval → apply
```

Sample Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }

        stage('Terraform Format') {
            steps {
                sh 'terraform fmt -check'
            }
        }

        stage('Terraform Validate') {
            steps {
                sh 'terraform validate'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan -out=tfplan'
            }
        }
    }
}
```

```

    stage('Approval') {
        steps {
            input message: 'Approve Terraform Apply?'
        }
    }

    stage('Terraform Apply') {
        steps {
            sh 'terraform apply -auto-approve tfplan'
        }
    }
}

```

LAB 29: GitHub Actions Pipeline Concept

Folder:

```
.github/workflows/terraform.yml
```

Example:

```

name: Terraform Pipeline

on:
  push:
    branches:
      - main
  pull_request:

jobs:
  terraform:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init

      - name: Terraform Format

```

```
run: terraform fmt -check

- name: Terraform Validate
  run: terraform validate

- name: Terraform Plan
  run: terraform plan
```

LAB 30: Terraform Security Scanning

Tools

Tool	Purpose
TFLint	Code linting
Checkov	IaC security scan
tfsec	Terraform security scan
Trivy	IaC/container scan
Terratest	Infrastructure testing

TFLint Example

```
tflint --init
tflint
```

Checkov Example

```
checkov -d .
```

Trivy IaC Scan

```
trivy config .
```

LAB 31: Common Errors and Fixes

Error 1: Provider Not Installed

Fix:

```
terraform init
```

Error 2: Azure Login Expired

Fix:

```
az logout  
az login
```

Error 3: Storage Account Name Already Taken

Fix:

Use globally unique name.

Example:

```
name = "satishstorage20260520"
```

Error 4: Resource Group Not Found

Reason:

Backend resource group or resource group does not exist.

Fix:

Create backend resource group first.

Error 5: Backend Authentication Issue

Fix options:

```
az login
az account set --subscription "SUBSCRIPTION_ID"
```

Or use storage account key.

Error 6: for_each Number Error

Wrong:

```
for_each = toset([1, 2, 3])
```

Correct:

```
for_each = toset(["1", "2", "3"])
```

LAB 32: Interview-Ready Terraform Explanation

Basic Answer

Terraform is an Infrastructure as Code tool used to create, modify, and delete infrastructure using declarative configuration files.

Strong Interview Answer

Terraform is an Infrastructure as Code tool that allows teams to provision and manage cloud infrastructure in a repeatable, version-controlled, and automated way. It uses providers to interact with platforms like Azure, AWS, GCP, Kubernetes, and many others. Terraform maintains a state file to track real infrastructure and compare it with desired configuration. In production, Terraform is commonly integrated with CI/CD pipelines, remote backend, state locking, security scanning, approval workflows, and modular reusable code.

LAB 33: Final Practice Assignment

Create a complete Terraform project with:

1. Resource Group
2. Storage Account
3. Storage Container
4. Variables

5. tfvars
 6. Outputs
 7. Remote backend
 8. Tags
 9. Lifecycle prevent_destroy
 10. Module-based structure
-

LAB 34: Final Project Example Structure

```
final-terraform-project/  
├─ backend.tf  
├─ providers.tf  
├─ main.tf  
├─ variables.tf  
├─ terraform.tfvars  
├─ outputs.tf  
└─ modules/  
    ├─ resource_group/  
    └─ storage_account/
```

LAB 35: Final Checklist

Before running Terraform in production, check:

- Code formatted
 - Code validated
 - Remote backend configured
 - State locking enabled
 - Secrets not hardcoded
 - tfvars reviewed
 - Plan reviewed
 - Security scan completed
 - Approval taken
 - Apply executed through pipeline
 - Outputs verified
-

Terraform Command Cheat Sheet

```
terraform version  
terraform init  
terraform fmt  
terraform fmt -check
```

```
terraform validate
terraform plan
terraform plan -out=tfplan
terraform apply
terraform apply tfplan
terraform apply -auto-approve
terraform destroy
terraform destroy -auto-approve
terraform output
terraform show
terraform state list
terraform state show RESOURCE_NAME
terraform state rm RESOURCE_NAME
terraform import RESOURCE_NAME RESOURCE_ID
terraform workspace list
terraform workspace new dev
terraform workspace select dev
```

Final Summary

Terraform is used in real companies for cloud automation, landing zone deployment, Kubernetes infrastructure, CI/CD infrastructure provisioning, compliance, security, and multi-cloud automation.

To become expert, practice in this order:

1. Basic resources
2. Variables and outputs
3. State and backend
4. Lifecycle
5. count and for_each
6. Modules
7. Import and drift
8. CI/CD pipeline
9. Security scanning
10. Enterprise folder structure